

MACHINE EXERCISE 1: RED LIGHT GREEN LIGHT (RLGL)

1. ALGORITHM DESIGN

The application reads the webcam stream continuously and resizes the incoming frames to a width of 640 pixels to ensure high-performance processing. To simplify the data, the frames are then converted to grayscale. To detect movement, the algorithm computes the absolute pixel-wise difference between the current grayscale frame and the previous grayscale frame using the formula `diff = abs(current_gray - previous_gray)`. The mean of these differences is calculated and normalized to yield the final score using `motion_score = mean(diff) / 255.0`. An Exponential Moving Average (EMA) is applied to smooth the UI display, while the raw score is used for strict game logic evaluation.

2. STATE MACHINE DESIGN

The game loop operates on a finite state machine that incorporates five distinct states: GREEN, RED, WARNING, DEAD, and LEVEL_UP.

- GREEN: The player must maintain a motion score above the `green_move_threshold`. If the player remains still and the idle timer exceeds the `idle_warning_ms` limit, it triggers the WARNING state.
- WARNING: A grace period prompting the player to move. Failure to move before the timer exceeds `idle_death_ms` shifts the state to DEAD. After the green phase timer expires, the state switches to RED.
- RED: The player must freeze. After an initial `red_grace_ms` period, any motion exceeding the `red_move_threshold` results in a DEAD state. If the timer expires without a violation, the game continues successfully.
- DEAD: Game over state triggered by rule violations.
- LEVEL_UP: A transitional state triggered after surviving a set number of cycles.

3. PARAMETER VALUES USED

The baseline parameters utilized for Level 1 logic are as follows:

- `green_move_threshold`: 0.04
- `red_move_threshold_base`: 0.055
- `red_grace_ms`: 650
- `idle_warning_ms`: 1800
- `idle_death_ms`: 3600

- `green_duration`: Randomized between 2600ms and 4200ms
- `red_duration`: Randomized between 1700ms and 2900ms

4. OBSERVED FAILURE CASES & MITIGATIONS

- a. **Low Contrast / Camouflage:** If the player's clothing or skin tone closely matches the background color (especially when both are converted to grayscale), physical movement will not result in a significant change in pixel values. The frame difference calculation will remain near zero, causing the system to fail to detect actual movement. This can result in an unfair "Death" from the idle timer during a GREEN state.
 - *Mitigation:* The simplest mitigation is standardizing the play area by instructing the player to ensure they are playing against a highly contrasting background. Future iterations could use pose estimation to track structural features rather than purely relying on pixel luminance.
- b. **Sudden Lighting Changes:** A sudden shift in environmental lighting (such as a lamp turning on or a cloud passing by a window) rapidly changes the pixel values across the entire frame. The simple frame-difference algorithm interprets this global pixel change as massive player movement, causing a false failure during a RED light.
 - *Mitigation:* The game must be played in an environment with stable, artificial lighting. Future improvements could involve dynamic background subtraction or computing motion strictly within a cropped bounding box around the player's face to ignore environmental light shifts.
- c. **Proximity to Camera:** If the player stands very far from the camera, large physical movements only change a small number of pixels, resulting in a low motion score. Conversely, if the player is too close, subtle movements like breathing might trigger a massive pixel shift and exceed the RED threshold.
 - *Mitigation:* Establish a baseline physical distance for the player (e.g., framing the player from the shoulders up) so the pixel-change-to-physical-movement ratio remains relatively consistent with the programmed thresholds.